

01 / Syntax, Blöcke & Kommentare

Code strukturieren, Gültigkeitsbereiche unterscheiden

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

01-01 PHP-Block, Anweisungen, Klammern

<?php startet PHP. Zuweisungen/Aufrufe enden mit ;. () = Bedingungen/Argumente, {} = Codeblock, [] = Array oder Zugriff. Reine PHP-Dateien ohne ?> beenden.

```
<?php
$alter = 20;
$namen = ["Mia", "Tom"];
if ($alter >= 18) {
    echo $namen[0];
}
```

01-02 Geschweifte Klammern ≠ eigener Scope

Scope = Gültigkeitsbereich einer Variablen. {} fasst Anweisungen zusammen. In PHP erzeugen if und Schleifen **keinen eigenen Variablen-Scope**. Die Variable unten bleibt nach dem Block verfügbar.

```
if (true) {
    $name = "Mia";
}
echo $name; // Mia, auch nach dem Block
```

01-03 Funktionen haben einen lokalen Scope

Parameter und normale Funktionsvariablen sind **lokal**. Eine gleichnamige Variable außerhalb ist davon getrennt. Benötigte Werte als Argumente hineinreichen und mit return zurückgeben.

```
$name = "Mia";
function zeigeName() {
    $name = "Tom"; // lokal
    echo $name;
}
zeigeName(); // Tom
echo $name; // Mia
```

01-04 Grundlegende Datentypen

Der zugewiesene Wert bestimmt den Variablentyp. int = Ganzzahl, float = Gleitkommazahl, string = Text, bool = Wahrheitswert. Dezimalpunkt statt Komma; null bedeutet „kein Wert“.

```
$zahl = 7; // int
$preis = 4.50; // float
$name = "Mia"; // string
$aktiv = true; // bool
$werte = []; // array
$fehlt = null; // null
```

01-05 Einzeilig: // und

Gleiche Kommentarwirkung: bis Zeilenende oder bis ?>. // stammt aus dem C++-Stil, # aus dem Shell-Stil. Besonderheit ab PHP 8: # **beginnt ein Attribut**, keinen normalen Kommentar.

```
// Kommentar im C++-Stil
# Kommentar im Shell-Stil
$anzahl = 3; // Kommentar nach Code
# [Hinweis] Mit Leerzeichen: Kommentar
```

Ergänzende Referenz: [Attribute](#)

01-06 Blockkommentar: /* ... */

Kann eine oder mehrere Zeilen umfassen; endet am ersten */. **Nicht verschachteln**. Ein normaler Blockkommentar ist **nicht automatisch ein PHPDoc-Kommentar**.

```
/* Ein kurzer Kommentar */
/* Mehrere Zeilen:
   Hier wird der Ablauf erklärt.
*/
echo "Weiter";
```

01-07 Dokumentationskommentar: /** ... */

Zwei Sterne am Anfang kennzeichnen den DocBlock für IDEs und Dokumentationswerkzeuge. @param beschreibt Parameter, @return die Rückgabe. Dokumentation erzwingt keine PHP-Typprüfung.

```
/**
 * Verdoppelt eine Zahl.
 * @param int $zahl Eingabewert
 * @return int Ergebnis
 */
function verdoppeln($zahl) {
    return $zahl * 2;
}
```

01-08 Ausgabe & Fehlersuche

echo/print geben aus. var_dump zeigt Wert und Typ; print_r macht Arrays lesbar. echo false erzeugt keinen sichtbaren Text. Für die Prüfung von Wahrheitswerten var_dump nutzen.

```
echo "Hallo";
print " Welt";
var_dump(7, "7", false);
print_r(["name" => "Mia"]);
```

02 / Bezeichner & Typumwandlung

Sprachregeln, Namenskonventionen und Konvertierung

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

02-01 Bezeichner: Was ist syntaktisch erlaubt?

Für einfache Namen: zuerst Buchstabe oder `_`, danach auch Ziffern. **Keine Leerzeichen oder Bindestriche**. Variablen beginnen mit `$`. Reservierte Wörter wie `class` nicht als Klassen-/Funktionsnamen verwenden.

```
$kunde2 = "Mia";
$_anzahl = 3;
// Ungültig: $2kunde, $vor-name
// Ungültig: $vor name
```

PHP erlaubt auch Nicht-ASCII-Zeichen in Namen; hier verwenden wir einfache ASCII-Bezeichner.

02-02 Groß-/Kleinschreibung beachten

Variablen und Konstanten unterscheiden die Schreibweise. Klassen-/Funktionsnamen sind sprachseitig nicht case-sensitiv. Trotzdem immer gleich schreiben; beim Autoloading kann die Schreibweise entscheidend sein.

```
$name = "Mia";
$Name = "Tom"; // andere Variable
const LIMIT = 10;
echo LIMIT;
// echo limit; // Fehler in PHP 8
```

Ergänzende Referenz: [Konstanten](#) · [Klassen](#) · [Funktionen](#)

02-03 Schreibweisen als Konvention

Konvention, keine allgemeine Syntaxpflicht: Konstanten in UPPER_SNAKE_CASE, Klassen in UpperCamelCase, Variablen und eigene Funktionen hier in lowerCamelCase. PSR-1 sieht camelCase für Methoden vor.

```
const MAX_TEILNEHMER = 20;
class KursTeilnehmer {}
$anzahlTeilnehmer = 3;
function zeigeAnzahl() {}
```

UPPERCASE: alle Buchstaben groß. Unterstriche trennen die Wörter. Variablenamen sind eine Projektkonvention.

02-04 Konstanten: const und define

Kein \$, kein späteres Überschreiben. `const` ist eine Deklaration, `define()` ein Funktionsaufruf. Global definierte Konstanten können auch in Funktionen ohne `global` verwendet werden.

```
const MAX_PLAETZE = 20;
define("KURS_NAME", "PHP");
echo MAX_PLAETZE; // 20
echo KURS_NAME; // PHP
```

02-05 intval(): in eine Ganzzahl umwandeln

Der PHP-Funktionsname lautet `intval()`. Schneidet Nachkommastellen **Richtung 0** ab; rundet nicht. Der optionale zweite Parameter gibt die Zahlenbasis an und wirkt bei String-Eingaben.

```
intval("12"); // 12
intval(12.9); // 12
intval(-12.9); // -12
intval("101", 2); // 5 (binaer)
```

02-06 floatval(): in eine Gleitkommazahl umwandeln

Dezimalpunkt, kein Dezimalkomma. Ein numerischer Anfang kann genügen; nachfolgender Text wird ignoriert. Beginnt der Text nicht numerisch, entsteht hier `0.0`. Deshalb kein Ersatz für Validierung.

```
floatval("12.75"); // 12.75
floatval("12,75"); // 12.0
floatval("12.75 EUR"); // 12.75
floatval("EUR 12.75"); // 0.0
```

02-07 Casts, strval() und boolval()

(int), (float), (string) und (bool) wandeln explizit um. `strval` liefert einen String, `boolval` einen Wahrheitswert.

Der Text "false" ist wahr, "0" ist falsch.

```
$anzahl = (int) "12"; // 12
$preis = (float) "12.75"; // 12.75
$text = strval(12); // "12"
boolval("0"); // false
boolval("false"); // true
```

Ergänzende Referenz: [boolval](#)

02-08 Umwandeln ist nicht validieren

intval akzeptiert auch einen Zahlenanfang. `is_numeric` prüft Zahlen bzw. vollständige Zahlenstrings; `is_int` nur den Datentyp `int`. Ganzzahleingaben mit `FILTER_VALIDATE_INT` prüfen (Seite 9).

```
intval("12abc"); // 12
is_numeric("12abc"); // false
is_numeric("12.5"); // true
is_int("12"); // false
is_int(12); // true
```

03 / Operatoren & Entscheidungen

Zuweisen, vergleichen, rechnen und verzweigen

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

03-01 = zuweisen · == vergleichen · === prüfen

= speichert einen Wert. == vergleicht mit möglichen Typumwandlungen; === **prüft zusätzlich den Typ**. Vergleiche liefern bool. In Bedingungen nicht versehentlich = schreiben.

```
$zahl = 7;           // Zuweisung
var_dump(7 == "7");  // true
var_dump(7 === "7"); // false
var_dump(7 === 7);   // true
var_dump(7 === 7.0); // false
```

03-02 Ungleich, Grenzen & Typfallen

!= ist das Gegenteil von ==, !== von ===. <, >, <=, >= prüfen Grenzen. Zahlenstrings können numerisch verglichen werden. Ungeprüfte Mischtypen vermeiden.

```
var_dump(7 != "7");    // false
var_dump(7 !== "7");   // true
var_dump(18 >= 18);    // true
var_dump("02" == "2"); // true
var_dump(0 == "Text"); // false (PHP 8)
```

03-03 Rechnen & verkürzte Zuweisung

+, -, *, / rechnen; % liefert den ganzzahligen Rest, ** potenziert. +=, -=, *=, /= ändern die Variable. ++/-- erhöhen/senken um 1; vor/nach der Variable unterscheidet sich.

```
echo 7 % 3;           // 1
echo 2 ** 3;          // 8
$n = 5;
echo $n++;            // 5; danach n = 6
echo ++$n;            // 7; danach n = 7
$n += 2;              // n = 9
```

03-04 UND, ODER, NICHT & Rangfolge

&&: beide wahr; ||: mindestens eine wahr; !: umkehren. &&|| werten rechts nur bei Bedarf aus. and/or binden schwächer als =; deshalb nicht blind austauschen. Kein not-Operator.

```
$alter = 20;
$ok = ($alter >= 18 && $alter <= 65);
var_dump(!$ok || $alter < 0); // false
$a = true && false;           // a = false
$b = true and false;         // b = true
```

03-05 Klammern, Rundung und Genauigkeit

() klärt die Rechenreihenfolge. round() rundet auf die gewünschte Anzahl Nachkommastellen. **Gleitkommazahlen sind nicht immer exakt**; intval() ist keine Rundungsfunktion.

```
echo (2 + 3) * 4;       // 20
echo round(12.75, 1);   // 12.8
echo round(0.1 + 0.2, 2); // 0.3
var_dump(0.1 + 0.2 === 0.3);
// false (Gleitkomma-Ungenauigkeit)
```

03-06 if · elseif · else

Die erste wahre Bedingung gewinnt; else ist der Restfall. Bedingungen mit ==/=== prüfen, nicht mit = zuweisen. Kein ; direkt hinter if (...).

```
$punkte = 70;
if ($punkte >= 80) {
    echo "Gut";
} elseif ($punkte >= 50) {
    echo "Bestanden";
} else {
    echo "Nicht bestanden";
}
```

03-07 switch · case · default

switch vergleicht wie ==, nicht wie ===. Ohne break läuft der nächste Fall weiter. default greift, wenn kein case passt; mehrere case können einen Block teilen.

```
$wahl = "1";
switch ($wahl) {
    case 1:
        echo "Start"; // trifft auch "1"
        break;
    default:
        echo "Unbekannt";
}
```

03-08 Ternärer Operator: ? :

Kurzform für eine Entscheidung zwischen zwei Werten: Bedingung ? WertBeiWahr : WertBeiFalsch. Das Ergebnis kann gespeichert oder ausgegeben werden.

```
$alter = 17;
$text = ($alter >= 18) ? "Erwachsen"
    : "Minderjährig";
echo $text; // Minderjährig
```

04 / Schleifen & Funktionen

Wiederholen, abbrechen und Code wiederverwenden

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

04-01 for: Start; Bedingung; Änderung

Start einmal ausführen, vor jedem Durchlauf prüfen, nach jedem Durchlauf ändern. < und <= unterscheiden sich um den eingeschlossenen Grenzwert.

```
for ($i = 1; $i <= 3; $i++) {
    echo $i . " "; // 1 2 3
}
// Hinter for (...) steht hier kein ;.
```

04-02 while & do ... while

while prüft vorher: auch null Durchläufe möglich. do ... while prüft danach: mindestens ein Durchlauf. Hier gehört hinter while (...) ein ;. Fortschritt/Abbruch sicherstellen.

```
$i = 3;
while ($i < 3) {
    echo $i++; // läuft nicht
}
do {
    echo $i++; // gibt einmal 3 aus
} while ($i < 3);
```

04-03 foreach: Werte oder Key und Wert

foreach durchläuft Arrays ohne Zähler; es funktioniert auch bei String-Keys und Indexlücken. Vorher bei unsicheren Daten is_array prüfen. => trennt Key und Wert.

```
$preise = ["Brot" => 3, "Milch" => 2];
foreach ($preise as $wert) {
    echo $wert . " "; // 3 2
}
foreach ($preise as $key => $wert) {
    echo "$key: $wert ";
}
```

04-04 continue · break · return

continue überspringt den Rest des Durchlaufs; break beendet die innerste Schleife bzw. switch. return beendet die Funktion. In switch innerhalb einer Schleife: continue 2 für die äußere Schleife.

```
for ($i = 1; $i <= 5; $i++) {
    if ($i === 2) { continue; }
    if ($i === 4) { break; }
    echo $i . " "; // 1 3
}
```

04-05 Verschachtelte Schleifen

Für jeden äußeren Durchlauf läuft die innere Schleife erneut. Zähler auseinanderhalten. break verlässt eine Ebene, break 2 verlässt zwei Ebenen.

```
for ($zeile = 1; $zeile <= 2; $zeile++) {
    for ($spalte = 1; $spalte <= 3; $spalte++) {
        echo " ";
    }
    echo "<br>"; // zweimal ***
}
```

04-06 Funktion, Parameter, Rückgabewert

Parameter sind lokal; ein Standardwert ist optional. Skalare/Arrays werden ohne & per Wert übergeben. return liefert das Ergebnis an den Aufruf; echo gibt nur aus.

```
function addiere($a, $b = 1) {
    return $a + $b;
}
$zahl = 4;
$summe = addiere($zahl, 3); // 7
echo addiere($zahl); // 5
// $zahl bleibt 4.
```

04-07 Typen & declare(strict_types=1)

declare(strict_types=1) muss die erste Anweisung der Datei sein. Strenge Skalartypen gelten für Aufrufe aus dieser Datei; int ist bei float erlaubt. **Ändert weder == noch ===.** : int ist der Rückgabetyt.

```
<?php
declare(strict_types=1);
function doppelt(int $n): int {
    return $n * 2;
}
echo doppelt(4); // 8
// doppelt("4"); // TypeError
```

04-08 Andere PHP-Dateien einbinden

include lädt eine Datei; bei Fehlschlag Warnung. require führt bei Fehlschlag zu einem Error. _once verhindert erneutes Einbinden. __DIR__ bezeichnet den Ordner dieser Datei.

```
// Datei funktionen.php muss existieren.
require_once __DIR__ . "/funktionen.php";
// Weitere Variante:
// include __DIR__ . "/kopf.php";
```

05 / Arrays: Keys & Zugriffe

Anlegen, lesen, vergleichen und Schlüssel verstehen

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

05-01 Ein Array: geordnete Key-Wert-Paare

Keys sind int oder string, Werte beliebige Typen. Numerische und assoziative Einträge dürfen gemischt werden. [] ist die Kurzform von array(); ohne Keys beginnt eine neue Liste bei 0.

```
$liste = ["Mia", "Tom"];
echo $liste[0]; // Mia
$person = ["name" => "Mia", "alter" => 20];
echo $person["name"]; // Mia
```

05-02 Leerer Key ist erlaubt; [] ist etwas anderes

"" ist ein gültiger String-Key. null als Key wird zu "". \$a[] hängt mit automatischem Integer-Key an – es bedeutet nicht „leerer String-Key“.

```
$a = [" " => "Leerkey"];
$a[] = "Angehängt"; // Key 0
$a[null] = "Neu"; // überschreibt ""
echo $a[" "]; // Neu
echo $a[0]; // Angehängt
```

05-03 Doppelte Keys: letzter Wert gewinnt

Im Quelltext darf derselbe Key mehrfach stehen. Im fertigen Array existiert er **nur einmal: Der letzte Wert gewinnt**. Gleiche Werte unter anderen Keys sind erlaubt.

```
$a = ["name" => "Mia", "name" => "Tom"];
echo $a["name"]; // Tom
echo count($a); // 1
$a["name"] = "Lena"; // ersetzt Tom
```

05-04 Key-Umwandlung kann Einträge überschreiben

"8" wird Key 8; "08" und "+8" bleiben Strings. true wird 1, false wird 0. Float-Keys werden zu int gekürzt; mit Nachkommaverlust ist das seit PHP 8.1 veraltet. int/string verwenden.

```
$a = [8 => "A", "8" => "B", "08" => "C"];
echo $a[8]; // B
echo $a["08"]; // C
$b = [1 => "Ja", true => "Neu"];
echo $b[1]; // Neu
```

05-05 String-Keys: Quotes & Großschreibung

Feste String-Keys in Anführungszeichen schreiben; Variablen-Keys nicht. "Name" und "name" sind verschieden. Ein fehlender Key erzeugt bei direktem Zugriff eine Warnung: vorher prüfen oder ?? nutzen.

```
$a = ["Name" => "Mia", "name" => "Tom"];
$key = "Name";
echo $a[$key]; // Mia
echo $a["name"]; // Tom
echo $a["ort"] ?? "?"; // ?
```

05-06 Anhängen, löschen, Indexlücken

Bei nichtnegativen Integer-Keys hängt [] hinter dem höchsten bisher verwendeten Key an, nicht bei count(). unset nummeriert nicht neu. array_values erstellt neue Keys 0, 1,

```
$a = [2 => "A", 5 => "B"];
unset($a[5]);
$a[] = "C"; // Key 6, nicht 1 oder 5
$a = array_values($a); // [0=>"A", 1=>"C"]
```

Versionsfall: Nach \$a = [-5 => "A"]; \$a[] = "B"; ist der neue Key ab PHP 8.3 -4, davor 0.

05-07 Verschachtelte Arrays

Ein Array kann weitere Arrays enthalten. Jede Klammer greift eine Ebene tiefer. Lesen und Ändern erfolgen über den vollständigen Key-Pfad.

```
$kunden = [
    ["name" => "Mia", "punkte" => 4],
    ["name" => "Tom", "punkte" => 2]
];
$kunden[0]["punkte"] += 3;
echo $kunden[0]["punkte"]; // 7
```

05-08 Array-Vergleich: Reihenfolge zählt bei ==

== prüft dieselben Key-Wert-Paare ohne gleiche Reihenfolge und mit lockeren Wertvergleichen. === verlangt zusätzlich gleiche Reihenfolge und Typen.

```
$a = ["x" => 1, "y" => 2];
$b = ["y" => 2, "x" => 1];
var_dump($a == $b); // true
var_dump($a === $b); // false
```

06 / Arrays: Ändern & Auswerten

Werte verändern, suchen, sortieren und zusammenführen

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

06-01 Werte in foreach wirklich ändern

Ohne & ist \$wert kein Verweis auf den Array-Eintrag. Nur \$wert zu ändern ändert das ursprüngliche Array nicht. Über den Key gezielt zurückschreiben.

```
$punkte = ["Mia" => 4, "Tom" => 2];
foreach ($punkte as $key => $wert) {
    $punkte[$key] = $wert + 1;
}
// ["Mia" => 5, "Tom" => 3]
```

06-02 Kopie vs. Referenz: & bewusst einsetzen

\$b = \$a kopiert Array-Werte (ohne vorhandene Referenzen). &\$wert in foreach verändert die Originalwerte. **Danach unset(\$wert)**, sonst bleibt der Verweis auf den letzten Eintrag bestehen.

```
$a = [1, 2];
$b = $a;
$b[0] = 9; // $a bleibt [1, 2]
foreach ($a as &$wert) {
    $wert *= 2;
}
unset($wert); // $a ist [2, 4]
```

06-03 count · array_sum · min · max

count zählt Einträge, nicht den höchsten Key. Standardmäßig wird nur die äußere Ebene gezählt. Vor count bei unsicheren Daten is_array prüfen. sum/min/max hier auf nichtleere Zahlenarrays anwenden.

```
$zahlen = [2, 5, 3];
echo count($zahlen); // 3
echo array_sum($zahlen); // 10
echo min($zahlen); // 2
echo max($zahlen); // 5
```

06-04 Anhängen & an den Enden entfernen

[]/array_push hängen hinten an; array_unshift vorne. array_pop entfernt/liefert den letzten, array_shift den ersten Wert. shift/unshift nummerieren Integer-Keys neu; String-Keys bleiben.

```
$a = ["B"];
array_unshift($a, "A"); // ["A", "B"]
array_push($a, "C", "D");
$hinten = array_pop($a); // "D"
$vorne = array_shift($a); // "A"
// $a ist jetzt ["B", "C"].
```

06-05 array_keys · array_values

array_keys liefert die Schlüssel, array_values die Werte. Beide erzeugen neue Listen mit Keys ab 0. Ergebnis zuweisen; array_values allein ändert das Original nicht.

```
$a = ["rot" => 3, "blau" => 5];
$keys = array_keys($a); // ["rot", "blau"]
$werte = array_values($a); // [3, 5]
```

06-06 in_array · array_search: streng suchen

in_array liefert bool, array_search den Key oder false. Der dritte Parameter true aktiviert strenge Typprüfung. **Key 0 ist ein Treffer:** Ergebnis mit !== false prüfen.

```
$a = ["Mia", "Tom"];
var_dump(in_array("Mia", $a, true)); // true
$key = array_search("Mia", $a, true);
if ($key !== false) {
    echo $key; // 0
}
```

06-07 sort · asort · ksort

sort: Werte sortieren, Keys neu. asort: Werte sortieren, Keys behalten. ksort: nach Keys sortieren. Absteigend: rsort/arsort/ksort. **Ändern das Array direkt;** Rückgabe nicht als Array speichern.

```
$a = ["b" => 1, "a" => 3];
$b = $a; $c = $a;
sort($a); // [0=>1, 1=>3]
asort($b); // ["b"=>1, "a"=>3]
ksort($c); // ["a"=>3, "b"=>1]
```

06-08 array_merge vs. +

array_merge: bei gleichen String-Keys gewinnt rechts; Integer-Keys werden neu nummeriert, Werte angehängt. + bildet die Vereinigung: gleiche Keys behalten links ihren Wert; keine Neunummerierung.

```
$a = ["x" => 1, 0 => "A"];
$b = ["x" => 2, 0 => "B"];
$m = array_merge($a, $b);
// ["x"=>2, 0=>"A", 1=>"B"]
$u = $a + $b;
// ["x"=>1, 0=>"A"]
```

07 / Arrays & Zeichenketten

Links: Arrays umformen · Rechts: Textgrundlagen

UTF-8-Beispiele: mb_* benötigt mbstring. Rückgaben zuweisen; Funktionsaufrufe geben nicht automatisch aus.

07-01 array_slice vs. array_splice

slice liefert einen Ausschnitt, Original bleibt. splice entfernt/ersetzt im Original und liefert entfernte Werte. Positionen beginnen bei 0. Integer-Keys werden hier neu nummeriert.

```
$a = ["A", "B", "C"];
$teil = array_slice($a, 1, 1); // ["B"]
$weg = array_splice($a, 1, 1, ["X"]);
// $weg: ["B"]; $a: ["A", "X", "C"]
```

07-02 array_unique: doppelte Werte entfernen

Entfernt doppelte Werte, nicht doppelte Keys. Der erste passende Eintrag behält seinen Key. Standardmäßig erfolgt der Vergleich als String. array_values schließt anschließend Indexlücken.

```
$a = ["A", "A", "B"];
$neu = array_unique($a);
// [0=>"A", 2=>"B"]
$neu = array_values($neu); // ["A", "B"]
```

07-03 array_filter: nach eigener Regel auswählen

Die Prüffunktion entscheidet, welche Werte bleiben; Keys bleiben erhalten. Ohne Prüffunktion verschwinden auch 0 und "0". Ergebnis zuweisen, Original bleibt unverändert.

```
function nichtLeer($wert) {
    return $wert !== "";
}
$a = ["", "0", "Mia"];
$neu = array_filter($a, "nichtLeer");
// [1=>"0", 2=>"Mia"]
```

07-04 array_map: jeden Wert umformen

Ruft die Funktion für jeden Wert auf und erzeugt ein neues Array aus den Rückgaben. Bei genau einem Eingabearray bleiben die Keys erhalten.

```
function malZwei($wert) {
    return $wert * 2;
}
$a = ["x" => 3, "y" => 5];
$neu = array_map("malZwei", $a);
// ["x" => 6, "y" => 10]
```

07-05 Quotes, Variablen, . und .=

"..." setzt Variablen ein; '...' nicht. . verbindet, .= hängt hinten an (nicht vorne). + ist kein Textverbinder. \n erzeugt in "..." einen Zeilenumbruch, im normalen HTML aber keinen sichtbaren Umbruch.

```
$name = "Mia";
echo "Hallo $name"; // Hallo Mia
echo 'Hallo $name'; // Hallo $name
$text = "Hallo";
$text .= " " . $name; // Hallo Mia
echo "\"Ja\"\\n"; // "Ja" + Umbruch
```

07-06 trim · ltrim · rtrim

Entfernen bestimmte Leer-/Steuerzeichen an beiden Rändern, nur links oder nur rechts. Kein Entfernen innerer Leerzeichen. Rückgabe speichern: trim verändert die Variable nicht selbst.

```
$text = " Hallo Welt ";
$sauber = trim($text); // "Hallo Welt"
ltrim($text); // "Hallo Welt "
rtrim($text); // " Hallo Welt"
// $text ist weiterhin unverändert.
```

07-07 strlen zählt Bytes, mb_strlen Zeichen

strlen zählt Bytes, mb_strlen Zeichen. Für UTF-8 die Kodierung angeben. Umlaute brauchen mehrere Bytes; zusammengesetzte Emoji können mehrere Zeichen zählen. mb_* benötigt die Erweiterung mbstring.

```
$text = "Käse";
echo strlen($text); // 5 Bytes
echo mb_strlen($text, "UTF-8"); // 4 Zeichen
```

07-08 Groß- und Kleinschreibung

strtolower/strtoupper für ASCII; für Umlaute mb_strtolower/mb_strtoupper mit UTF-8 verwenden. Auch hier wird eine neue Zeichenkette zurückgegeben.

```
echo strtolower("HALLO"); // hallo
echo strtoupper("Hallo"); // HALLO
echo mb_strtolower("MÜLLER", "UTF-8");
// müller
echo mb_strtoupper("Ärger", "UTF-8");
// ÄRGER
```

08 / Textverarbeitung & Formulare

Text suchen und zerlegen, HTML ausgeben, Formulare senden

UTF-8-Beispiele: mb_* benötigt mbstring. Rückgaben zuweisen; Funktionsaufrufe geben nicht automatisch aus.

08-01 substr · mb_substr: Teilttext

Startposition ab 0, danach optionale Länge. Negative Startposition zählt vom Ende. substr arbeitet mit Bytes; für UTF-8-Zeichen mb_substr verwenden.

```
echo substr("Berlin", 0, 3); // Ber
echo substr("Berlin", -3); // lin
echo mb_substr("Käse", 1, 2, "UTF-8");
// äs
```

08-02 str_replace · str_ireplace

Ersetzen alle Vorkommen; ein leerer Ersatz löscht Treffer. str_replace beachtet Groß-/Kleinschreibung, str_ireplace ignoriert sie bei ASCII. Ergebnis zuweisen.

```
$text = "rot, rot, blau";
$text = str_replace("rot", "grün", $text);
// grün, grün, blau
echo str_replace("-", "", "AB-12"); // AB12
echo str_ireplace("php", "JS", "PHP php");
// JS JS
```

08-03 strpos: Position 0 ist kein false

Liefert die erste Byteposition oder false. 0 ist ein Treffer: deshalb !== false, nicht nur if (\$position). strpos ignoriert die Groß-/Kleinschreibung bei ASCII.

```
$position = strpos("PHP lernen", "PHP");
if ($position !== false) {
    echo $position; // 0 = Treffer am Anfang
}
var_dump(strpos("PHP", "Java")); // false
```

08-04 Enthalten? Anfang? Ende? (PHP 8)

str_contains, str_starts_with und str_ends_with liefern bool; Groß-/Kleinschreibung zählt. Bei leerem Suchtext liefern alle drei true.

```
var_dump(str_contains("Hallo", "all"));
// true
var_dump(str_starts_with("Hallo", "Ha"));
// true
var_dump(str_ends_with("datei.php", ".php"));
// true
```

08-05 explode · implode: Text ↔ Array

explode zerlegt anhand eines Trenntexts; dieser darf nicht leer sein. Wiederholte/abschließende Trennzeichen erzeugen leere Einträge. implode verbindet Werte; Keys werden nicht ausgegeben.

```
$teile = explode(";", "A;B;");
// ["A", "", "B", ""]
$text = implode(" ", ["Mia", "Tom"]);
// "Mia, Tom"
```

08-06 HTML-Zeichen ausgeben, Tags entfernen

htmlspecialchars maskiert Sonderzeichen für HTML-Text und zitierte Attribute. strip_tags entfernt Tags, ersetzt aber kein sicheres HTML-Escaping. Nicht als Schutz für JavaScript-/CSS-Kontexte verwenden.

```
$text = "<b>Mia</b>";
echo htmlspecialchars(
    $text, ENT_QUOTES, "UTF-8"
); // &lt;b&gt;Mia&lt;/b&gt;
echo strip_tags($text); // Mia
```

08-07 Formular: name wird zum PHP-Key

HTML

action bestimmt das Ziel; method die Übertragung. name benennt den gesendeten Key, id allein genügt nicht. GET-Daten stehen in der URL, POST-Formulardaten im Request-Body.

```
<form action="auswerten.php" method="post">
    <input name="name" type="text">
    <input name="menge" type="number" min="0">
    <button type="submit">Senden</button>
</form>
```

08-08 GET / POST auslesen & Methode prüfen

\$_GET/\$_POST sind Arrays. Einzelne Formularwerte kommen als Strings, auch type="number". POST verschlüsselt nicht automatisch: HTTPS nutzen. Browserprüfung ersetzt keine Serverprüfung.

```
$name = $_POST["name"] ?? "";
// Bei method="get": $_GET["name"] ?? ""
$methode = $_SERVER["REQUEST_METHOD"] ?? "";
if ($methode === "POST") {
    echo "Formular empfangen";
}
```

09 / Formularwerte sicher prüfen

Fehlend, null, leer und 0 bewusst unterscheiden

Beispiele einzeln im PHP-Block lesen. // nennt Ergebnis/Ausgabe; ohne echo/var_dump keine sichtbare Ausgabe.

09-01 Checkboxes & mehrere Werte

HTML

Nicht angekreuzte Checkboxes werden nicht gesendet; deaktivierte Felder ebenso wenig. `name="..."` sammelt Werte in einem Array. Formularwerte können deshalb auch Arrays statt Strings sein.

```
<input type="checkbox" name="extras[]"
      value="fruehstueck">
<input type="checkbox" name="extras[]"
      value="parkplatz">
<!-- Beide gewählt: 2 Werte in extras. -->
```

09-02 Array-Eingaben vor foreach prüfen

?? garantiert keinen Datentyp. Auch bei erwarteten Arrays `is_array` prüfen; enthaltene Werte getrennt prüfen. Nur erlaubte Auswahlwerte akzeptieren, nicht beliebige Request-Daten.

```
$extras = $_POST["extras"] ?? [];
$erlaubt = ["fruehstueck", "parkplatz"];
if (!is_array($extras)) { $extras = []; }
foreach ($extras as $extra) {
    if (in_array($extra, $erlaubt, true)) {
        echo $extra . " ";
    }
}
```

09-03 isset vs. array_key_exists

`isset` ist wahr bei vorhandenem Wert **ungleich null**. `array_key_exists` prüft nur den Key, auch bei Wert null. Fehlender Key: beide falsch. `isset` erkennt 0 und "" als gesetzt.

```
$a = ["name" => null, "menge" => 0];
var_dump(isset($a["name"])); // false
var_dump(array_key_exists("name", $a));
// true
var_dump(isset($a["menge"])); // true
```

09-04 empty: auch 0 und "0" sind „leer“

`empty` ist wahr bei fehlend, null, false, 0, 0.0, "", "0" und []. **Nicht einfach das Gegenteil von `isset`**. Ein Leerzeichen und "false" gelten als wahr/nicht leer.

```
var_dump(empty("0")); // true
var_dump(empty(0)); // true
var_dump(empty([])); // true
var_dump(empty(" ")); // false
var_dump(empty("false")); // false
```

09-05 ?? vs. ?: als Ersatzwert

?? greift nur bei fehlend/null und warnt nicht bei fehlendem Key. ?: greift bei allen falsy-Werten, also auch "0". Ein vorhandener Leerstring bleibt bei ?? erhalten.

```
$a = ["menge" => "0", "name" => ""];
echo $a["menge"] ?? "fehlt"; // 0
echo $a["menge"] ?: "leer"; // leer
echo $a["name"] ?? "Gast"; // Leerstring
echo $a["ort"] ?? "Berlin"; // Berlin
```

09-06 Text prüfen, ohne gültige 0 zu verlieren

Erst `is_string` prüfen, dann `trim`. && verhindert den `trim`-Aufruf bei einem Array. Der Vergleich `!== ""` lässt "0" zu. Nur Leerzeichen werden als leere Eingabe abgewiesen.

```
$roh = $_POST["name"] ?? "";
if (is_string($roh) && trim($roh) !== "") {
    $name = trim($roh);
} else {
    $name = "Gast";
}
```

09-07 Ganzzahl validieren, nicht nur casten

`FILTER_VALIDATE_INT` liefert int oder false. Deshalb `!== false: 0 ist gültig`. Danach den erlaubten Bereich prüfen. (int) "12abc" bzw. `intval("12abc")` ergäbe 12 – keine Validierung.

```
$roh = $_POST["menge"] ?? "";
$menge = filter_var($roh,
    FILTER_VALIDATE_INT);
if ($menge !== false && $menge >= 0) {
    echo $menge; // auch 0 ist erlaubt
} else {
    echo "Ungültige Menge";
}
```

09-08 Eingaben als Text im Browser ausgeben

Geprüften Text beim Einfügen in HTML maskieren. Für sichtbare Zeilenumbrüche erst `htmlspecialchars`, dann `nl2br`: sonst wird eingegebenes HTML ausgeführt oder `<br>` wieder maskiert.

```
$roh = $_POST["name"] ?? "";
$text = is_string($roh) ? trim($roh) : "";
$sicher = htmlspecialchars(
    $text, ENT_QUOTES, "UTF-8"
);
echo nl2br($sicher);
```